

The URL of the Web based remix IDE is remix.ethereum.org and it can be accessed directly from Web browsers for writing, compiling and executing the smart contracts.

In the following source code, a new currency MyCurrency is programmed using Solidity. The new currency with its own set of rules, protocols and security mechanism can be initialised during code customisation.

```
pragma solidity ^0.4.18;
contract MyCurrency {
    string public name = 'MyCurrency';
    // Name of the New Currency
    string public currencyName = 'Currency1.0';
    // Select Currency
    mapping (address => uint) balances;
    // Key-Value Pair for Address-Account
    event Transfer(address _sender, address _receiver, uint256
_value);
    // Log Recording
    constructor() public {
        // Constructor on Creating the Contract
        balances[msg.sender] = 100000;
        // Balance Confirmation
    }
    function sendAmount(address _receiver, uint _amount) public
returns(bool sufficient) {
    if (balances[msg.sender] < _amount) return false;
    // Authentication of the Transfer
    balances[msg.sender] -= _amount;
    balances[_receiver] += _amount;
    emit Transfer(msg.sender, _receiver, _amount);
    // Commit of Payment Transfer with Transaction Recording
    return true;
}
    function getBalance(address _addr) public view returns(uint)
{
    // Checking the Balance
    return balances[_addr];
}
}
```

On clicking on the *Deploy* option in the Remix IDE, the code is executed and detailed logs of the transaction can be viewed.

The detailed logs of the blockchain transaction can be analysed. There are multiple parameters in the transaction log including the gas limit, which in smart contract programming refers to the amount of work or throughput associated with the transaction.

NodeJS and Web3JS

NodeJS is a cross-platform open source platform for JavaScript based programming. It can be used for multiple applications including blockchain development,



1	pragma solidity >=0.4.22 <0.6.0;
[2] only remix transactions, script	
status	0x1 Transaction mined and execution succeed
transaction hash	0x1d4446d14a1e05325451c4b936c26bb8f181e16514183602f6a5276f5b22bdc2
contract address	0x692a70d2e424a56d2c6c27ea97d1a86395877b3a
from	0xca35b7d915458ef540ade6068dfc2f44e8fa733c
to	MyCurrency.(constructor)
gas	3000000 gas
transaction cost	616619 gas
execution cost	420635 gas
hash	0x1d4446d14a1e05325451c4b936c26bb8f181e16514183602f6a5276f5b22bdc2
input	0x608...00000
decoded input	{ "uint8 _numProposals": 0 }
decoded output	-
logs	[]

Figure 5: View logs and transaction details

smartphone applications, distributed Web applications, NoSQL processing, Big Data analytics, machine learning, Internet of Things (IoT) and many other topics related to advanced computing.

For blockchain programming, NodeJS is integrated with Web3JS, which is the set of libraries and tools for interaction with blockchain based connections.

The Web3JS platform for blockchains and dApp can be integrated with the following instructions:

```
npm: npm install web3
pure js: link the dist/web3.min.js
meteor: meteor add ethereum:web3
```

After installation, the code in JavaScript and Server Side Scripts is written with the smart contracts and deployed for the secured applications.

Truffle

Truffle is another programming suite for the development of blockchain based smart contracts that can be installed with NodeJS and be executed on Ethereum Virtual Machine (EVM).

With the execution of the following instruction, the Truffle suite is associated with NPM:

```
npm install -g truffle
```

After the installation of Truffle, the installed version can be checked in the terminal as follows:

```
truffle version
```